

《信息安全》第九讲

# 信息系统的容错技术

2023年11月13日

周亚建

yajian@bupt.edu.cn

# Overview of Last Class

信息系统  
可靠性

硬件  
可靠性

软件  
可靠性

网络  
可靠性

# 面对现实：如何实现系统的可靠性？



**避错**

**(Fault-avoidance)**

避免故障

通过对组成系统的部件进行严格的筛选、对系统进行严格的测试、对系统进行屏蔽以减少外界的干扰等方法来提高系统的可靠性。

容错技术与排错技术并不是相互对立的，它们可以相互补充，构成高可信的信息系统



**容错**

**(Fault-tolerance)**

发生故障后能  
正确运行

即使采用了排错技术，信息系统还是迟早会发生故障。因此在设计系统时应考虑一旦发生故障能自动检测出故障并使系统自动恢复正常运行。这样设计出来的系统在发生故障后仍能正常运行。

# 面对现实：如何实现系统的可靠性？





# 容错的概念

发展历史，各阶段的代表技术， .....

# 容错的概念



容忍故障，即故障一旦发生时能够**自动检测**出来并使系统能够**自动恢复正常运行**。

- 当出现某些指定的硬件故障或软件错误时，系统仍能执行规定的一组程序，或者说程序不会因系统中的故障而中止或被修改
- 执行结果也不包含系统中故障所引起的差错。

**容错设计的软件可以有某些规定数目的故障但不导致失效，但对无容错的软件而言，故障即失效。**



# 容错技术的起源



John von Neumann,  
1903~1957

匈牙利裔美籍数学家、  
计算机科学家

1952年，他在California技术学院做了关于容错技术研究的五个报告，他所提出的精辟论断成了以后容错技术研究的基础。

1956年，他发表了题为《概率逻辑及用不可靠元件设计可靠的结构》的论文，文中提出了多数表决的概念，并分析了这种结构对系统产生错误的概率及其可能产生的影响。这预示着容错计算方面的理论工作的开始。

## Probabilistic logics and the synthesis of reliable organisms from unreliable components

J. von Neumann

(Dated: 1956, Lectures delivered at the California Institute of Technology, January 1952)

In C. E. Shannon and J. McCarthy, editors, *Automata Studies*, pp. 329-378. Princeton University Press, Princeton, NJ, 1956.

### I. INTRODUCTION

The paper that follows is based on notes taken by Dr. R. S. Pierce on five lectures given by the author at the California Institute of Technology in January 1952. They have been revised by the author but they reflect, apart from minor changes, the lectures as they were delivered.

The subject-matter, as the title suggests, is the role of error in logics, or in the physics implementation of logics - in automata-synthesis. Error is viewed, therefore, not as an extraneous and misdirected or misdirecting accident, but as an essential part of the process under consideration - its importance in the synthesis of automata being fully comparable to that of the factor which is normally considered, the intended and correct logical structure.

Our present treatment of error is unsatisfactory and ad hoc. It is the author's conviction, voiced over many years, that error should be treated by thermodynamical methods, and be the subject of a thermodynamical theory, as information has been, by the work of L. Szilard and C. E. Shannon [Cf. VB]. The present treatment falls far short of achieving this, but it assembles, it is hoped, some of the building materials, which will have to enter into the final structure.

The author wants to express his thanks to K. A. Brueckner and M. Gell-Mann, then at the University of Illinois, to whose discussions in 1951 he owes some important stimuli on this subject; to Dr. R. S. Pierce at the California Institute of Technology, on whose excellent notes this exposition is based; and to the California Institute of Technology, whose invitation to deliver these lectures combined with the very warm reception by the audience, caused him to write this paper in its present form, and whose cooperation in connection with the present publication is much appreciated.

### II. A SCHEMATIC VIEW OF AUTOMATA

#### A. Logics and Automata

It has been pointed out by A. M. Turing [1] in 1937 and by W. S. McCulloch and W. Pitts [2] in 1943 that effectively constructive logics, that is, intuitionistic logics, can be best studied in terms of automata. Thus logical propositions can be represented as electrical networks or (idealized) nervous systems. Whereas logical propositions are built up by combining certain primitive symbols, net-

works are formed by connecting basic components, such as relays in electrical circuits and neurons in the nervous system. A logical proposition is then represented as a "black box" which has a finite number of inputs (wires or nerve bundles) and a finite number of outputs. The operation performed by the box is determined by the rules defining which inputs, when stimulated, cause responses in which outputs, just as a propositional function is determined by its values for all possible assignments of values to its variables.

There is one important difference between ordinary logic and the automata which represent it. Time never occurs in logic, but every network or nervous system has a definite time lag between the input signal and the output response. A definite temporal sequence is always inherent in the operation of such a real system. This is not entirely a disadvantage. For example, it prevents the occurrence of various kinds of more or less overt vicious circles (related to "non-constructivity", "impredicativity", and the like) which represent a major class of dangers in modern logical systems. It should be emphasized again, however, that the representative automaton contains more than the content of the logical proposition which it symbolizes - to be precise, it embodies a definite time lag.

Before proceeding to a detailed study of a specific model of logic, it is necessary to add a word about notation. The terminology used in the following is taken from several fields of science; neurology, electrical engineering, and mathematics furnish most of the words. No attempt is made to be systematic in the application of terms, but it is hoped that the meaning will be clear in every case. It must be kept in mind that few of the terms are being used in the technical sense which is given to them in their own scientific field. Thus, in speaking of a neuron, we don't mean the animal organ, but rather one of the basic components of our network which resembles an animal neuron only superficially, and which might equally well have been called an electrical relay.

#### B. Definitions of the Fundamental Concepts

Externally an automaton is a "black box" with a finite number of inputs and a finite number of outputs. Each input and each output is capable of exactly two states, to be designated as the "stimulated" state and the "unstimulated" state, respectively. The internal functioning of such a "black box" is equivalent to a prescription that specifies which outputs will be stimulated in response to

# 第一代和第四代系统的比较

| 项目              | UNIVAC I          | Tandem               | 提高                |
|-----------------|-------------------|----------------------|-------------------|
| 时间（年）           | 1951              | 1985-1987            |                   |
| 不可用性            | 0.172             | $2.8 \times 10^{-5}$ | $6.2 \times 10^7$ |
| MTTF（小时）        | 66                | $2.4 \times 10^3$    | $3.5 \times 10^3$ |
| 硬停机之间每次处理所执行指令数 | $4.7 \times 10^8$ | $2.6 \times 10^{17}$ | $5.4 \times 10^4$ |

闵应骅. 容错计算这二十五年[J]. 计算机学报, 1995, 18（12）： 930-943。



# 我国金融业为什么会进口这么多IBM的机器？

国内高端服务器市场销售额市场基本被IBM、HP、SGI、SUN等国外高性能服务器厂商垄断，其市场占有率达到95.6%。



闵应骅

中科院计算所研究员，博士生导师

中国计算机学会容错计算专业委员会主任

IEEE终身Fellow

IBM有一个承诺：系统失效，如果是属于软、硬件的问题，IBM包赔损失。虽然购买这样的服务费用很高。有这一条使用户很放心，而中国公司不肯做这样的承诺。IBM之所以敢这么承诺，首先是因为它对自己的产品有足够的信心。其次，它有很强、很高水平的服务团队，可以做到系统失效申报之后，2小时内响应，15分钟内到达。

闵应骅的博客：<http://blog.sciencenet.cn/home.php?mod=space&uid=290937&do=blog&quickforward=1&id=312932>

# 第四代计算机实例：80年代以来

2018年6月14日上午，习近平在济南考察了浪潮集团高端容错计算机生产基地。



高端容错计算机号称是金融和通信行业信息流动的“心脏”，是金融和通信关键业务的核心设备。由于某些领域的特殊性，必须要求计算机系统时刻保持工作状态，而意外总是不可避免的，所以容错计算机系统应运而生。

长期以来，我国的高端大型容错计算机系统都依赖国外进口。像银行业的核心业务在以前都是采用国外的计算机。

# 我国的容错计算机研究

2010年研制成功我国首台具有自主知识产权的高端容错计算机

- 突破“双翼可扩展多处理器紧耦合共享存储器体系结构”、“三级缓存两级目录一致性协议”、“**软硬一体化的容错技术体系**”等3项核心技术
- 研制1个核心部件——处理器协同芯片组
- 研制中国第一个通过Unix 03认证的Unix操作系统K-UX，建立了较为完整的自主技术体系



**浪潮天梭高端容错计算机**



浪潮集团首席科学家王恩东（2014年获奖）

可用度达到99.999%（即每年停机时间累计不超过5.26分钟）



# 容错的主要内容

故障检测与诊断，故障屏蔽，冗余，.....

# 容错技术的主要内容

故障检测和诊断技术

故障屏蔽技术

冗余技术





# 容错技术

## 故障检测和诊断

## 故障屏蔽技术

## 冗余技术

- ❑ 故障检测（Fault Detection）：判断系统是否存在故障的过程  
故障检测的作用是确认系统是否发生了故障，指示故障的状态，即查找故障源和故障性质。一般来说，故障检测只能找到错误点（错误单元），不能准确找到故障点。
- ❑ 故障诊断（Fault Diagnosis）：检测出系统存在故障后要进行故障的定位，找出故障所在的位置。



# 容错技术

**故障检测和诊断**

**故障屏蔽技术**

**冗余技术**

故障屏蔽技术是防止系统中的故障在该系统的信息结构中产生差错的各种措施的总称，其实质是在故障效应达到模块的输出以前，利用冗余资源将故障影响掩盖起来，达到容错目的。



扁鹊见蔡桓公，立有间，扁鹊曰：“君有疾在腠理，不治将恐深。”桓侯曰：“寡人无疾。”扁鹊出，桓侯曰：“医之好治不病以为功！”居十日，扁鹊复见，曰：“君之病在肌肤，不治将益深。”桓侯不应。扁鹊出，桓侯又不悦。居十日，扁鹊复见，曰：“君之病在肠胃，不治将益深。”桓侯又不应。扁鹊出，桓侯又不悦。居十日，扁鹊望桓侯而还走。桓侯故使人问之，扁鹊曰：“疾在腠理，汤熨之所及也；在肌肤，针石之所及也；在肠胃，火齐之所及也；在骨髓，司命之所属，无奈何也。今在骨髓，臣是以无请也。”居五日，桓侯体痛，使人索扁鹊，已逃秦矣。桓侯遂死。

——《韩非子·喻老》

# 容错技术

故障检测和诊断

故障屏蔽技术

冗余技术

硬件冗余

时间冗余

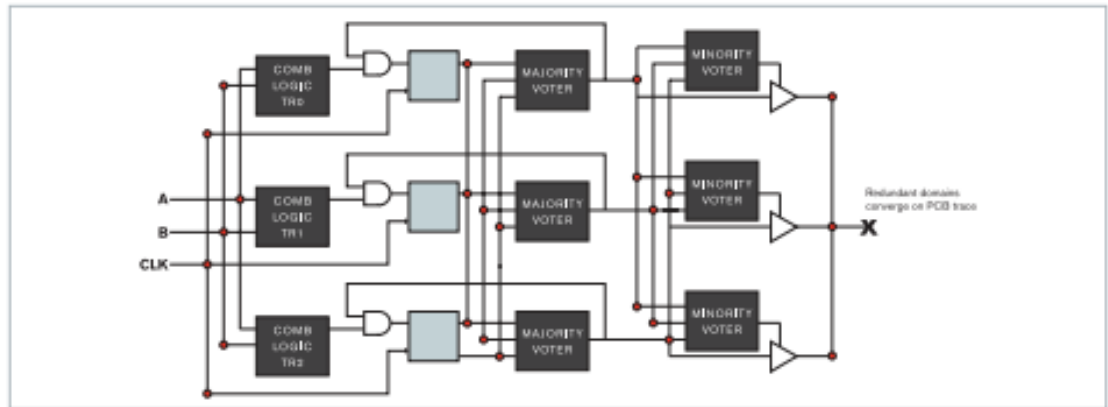
所谓冗余就是超过系统实现正常功能的额外资源。

软件冗余

信息冗余

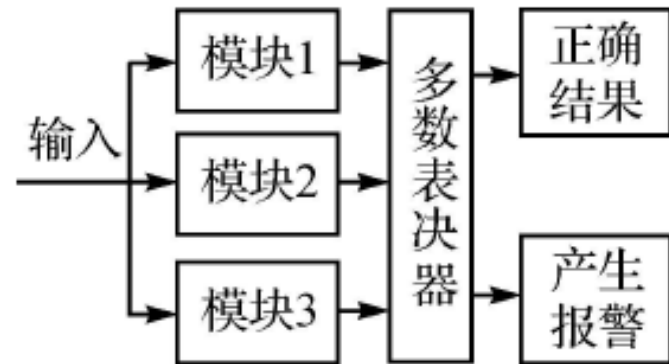
# 硬件冗余：三模冗余

- 三模冗余可以由用户手动完成，也可以依靠工具完成。Xilinx推出了XilInXtMrtool，可以方便地实现三模冗余设计，并且可以方便地配置被冗余项，大大方便了用户。



# 考试考！！！ 思考题

既然三模冗余（TMR）采用表决的方式容易造成判决错误，有无改进的方法？





# 时间冗余技术

**基本思想：**重复执行指令或者一段程序来消除故障的影响，以达到容错的效果，它是用消耗时间来换取容错的目的。

根据执行的是一条指令还是一段程序，分成两种方法：

## 指令复执

当检测出故障的时候，重复执行故障指令。若故障是瞬时的，则在指令复执期间可能不会出现，程序就可以继续向前运行。指令复执必须保留上一指令结束的现场（包括累加器、PC及其他状态寄存器的状态）。

## 程序卷回

它不是重复执行一条指令，而是重复执行一小段程序。在整段程序中可以设置多个恢复点，程序有错误的情况下可以从一个个恢复点处开始重复执行程序。首先检验一小段程序的计算结果，若结果出现错误则卷回再重复执行那个部分，若一次卷回不能解决，可以多次卷回，直到故障消除。



# 信息冗余技术

信息容错技术是通过在数据中附加冗余的信息位来达到故障检测和容错的目的。

- 通常情况下，附加的信息位越多，其检错纠错的能力就越强，但同时也增加了复杂度和难度。信息冗余最常见的有检错码和纠错码。
- 检错码只能检查出错误的存在，不能改正错误，而纠错码能检查出错误并能纠正错误。
- 常用的检错纠错码有奇偶校验码、海明码、循环码等。



# 信息冗余技术：奇偶校验

奇校验是通过增加一位校验位的逻辑取值，在源端将原数据代码中为1的位数形成奇数，然后在宿端使用该代码时，连同校验位一起检查为1的位数是否是奇数，做出进一步操作的决定。

偶校验道理与奇校验相同，只是将校验位连同原数据代码中为1的位数形成偶数。

- ❑ 奇偶校验只能检查一位错误，且没有纠错的能力。
- ❑ 奇偶校验器多设计成九位二进制数，以适应一个字节，一个ASCII代码的应用要求。

# 信息冗余技术：奇偶校验

- 信道编码概述
- Turbo及LDPC码
- 空时编码

## Polar码

- 网络编码

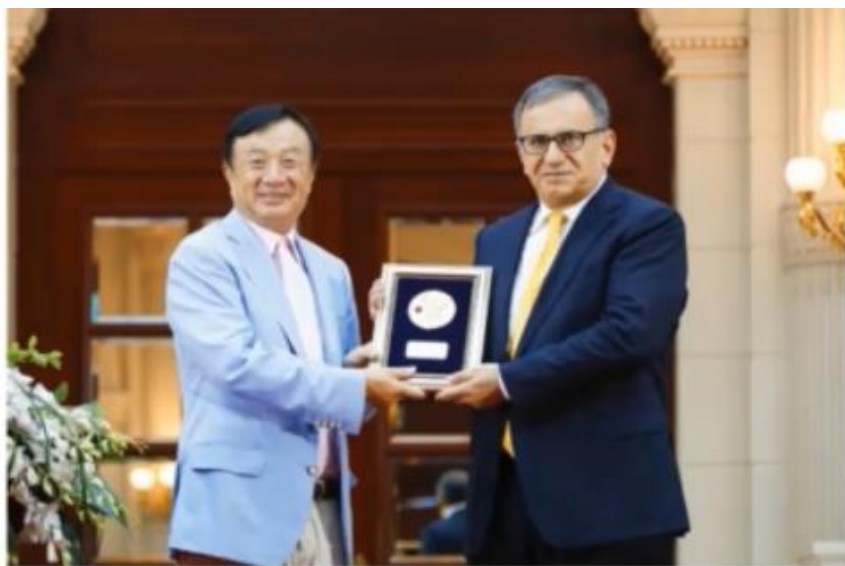


# 信息冗余技术：奇偶校验

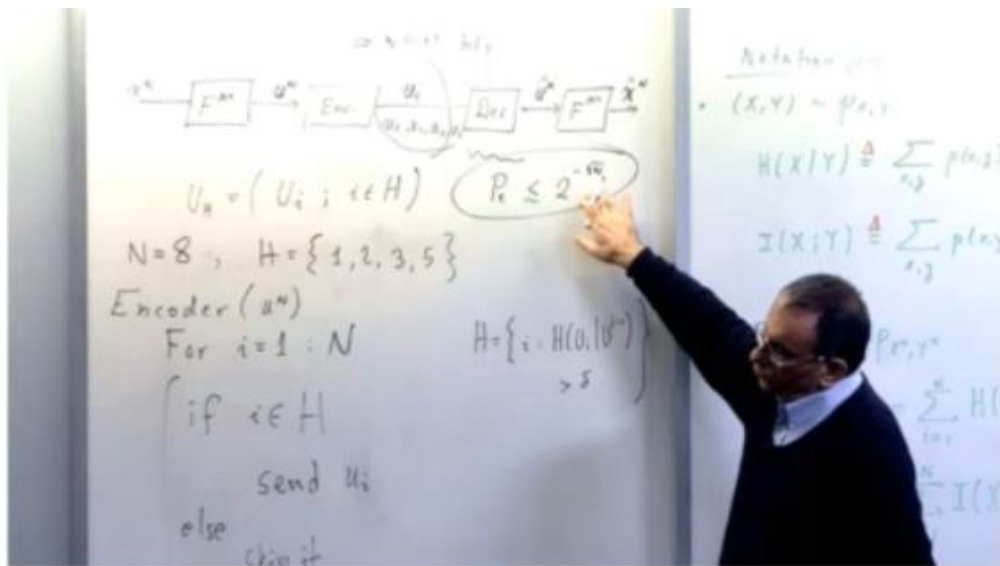
在二进制离散无记忆信道中，极化码是迄今为止唯一一种能够达到信道香农极限的编码方法，且其具有较低的编译码复杂度：对于长度为 $N$ 的数据块，极化码编码的复杂度为 $O(N\log N)$ ；其性能优势随着码长 $N$ 的增大而更加明显。极化码的提出是通信领域的重大突破，其本身亦成为信息论与编码领域备受瞩目的研究热点，发展前景广阔。

2010年，华为识别出polar码作为优秀信道编码技术的潜力，在Erdal Arikan教授研究基础上对其进行更深入的研究；其亦在极化码的核心原创技术上取得了多项突破。

2016年，在3GPP RAN1#87次会议上，国际移动通信标准化组织3GPP最终确定了5G eMBB（增强移动宽带）场景的信道编码技术方案：其以传统的LDPC码作为数据信道的编码方案，华为主导的Polar码被作为控制信道的编码方案。



华为创始人任正非向Erdal Arikan教授颁奖



Erdal Arikan教授讲解极化码

# 软件冗余技术

**软件容错是指在出现有限数目的软件故障的情况下，系统仍可提供连续正确执行的内在能力。其目的是屏蔽软件故障，恢复因出故障而影响的运行进程。**

**实现软件容错的基本方法，是将若干个根据同一需求说明编写的不同程序（即多版本程序），在不同空间同时运行，然后在每一个设置点通过表决或接收测试进行表决。**



# 最基本的软件容错技术

01

Algirdas Avizienis 提出的基于**静态冗余**的 $N$ 版本编程方法

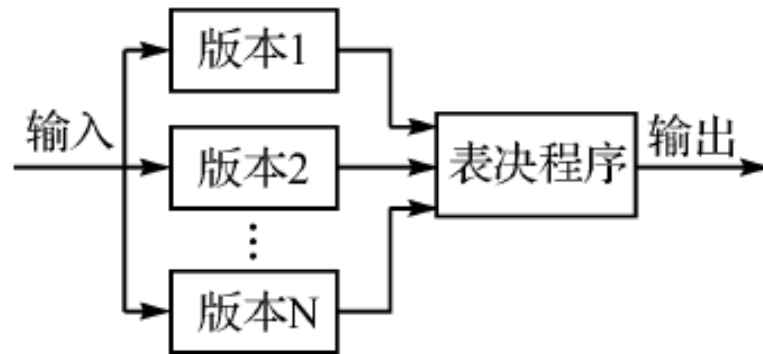
02

B. Randell 提出的基于**动态冗余**的恢复块技术



# 软件冗余技术： N-Version Programming 掌握！

- NVP（N-Version Programming）结构（多版本编程设计），由Algirdas Avizienis 于1977 年在“System Structure for Soft-ware Fault Tolerance”中提出。
- NVP是一种静态冗余方法，其基本设计思想是用 $N$ 个具有同一功能而采用不同编程方法的程序执行一项运算，其结果通过多数表决器输出。
- 这种容错结构方法有效避免了由于软件共性故障造成的系统出错，提高了软件的可靠性。



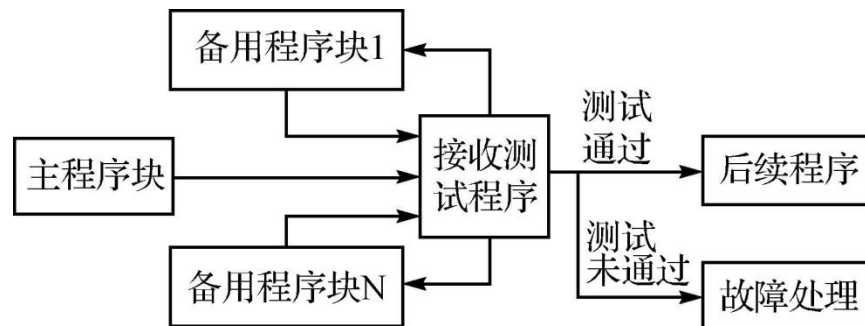
# 软件冗余技术: Recovery Block

## 掌握！

- RB (Recovery Block, 恢复块结构)，由Randell于1975年在“On The Implementation of *N*-Version Programming for Software Fault-Tolerance During Program Execution”中提出的一种的软件容错技术。
- RB是一种动态冗余方法。在RB结构中有主程序块和一些备用程序块，主程序块和备用程序块采用不同编程方法但具有相同的功能。每个主程序块都可以用一个根据同一需求说明设计的备用程序块替换。首先运行主程序块，然后进行接受测试，如果测试通过则将结果输出给后续程序；否则调用第一个备用块。依次类推，在 $N$ 个备用程序块替换完后仍没有通过测试，则要进行故障处理。



什么可靠性模型

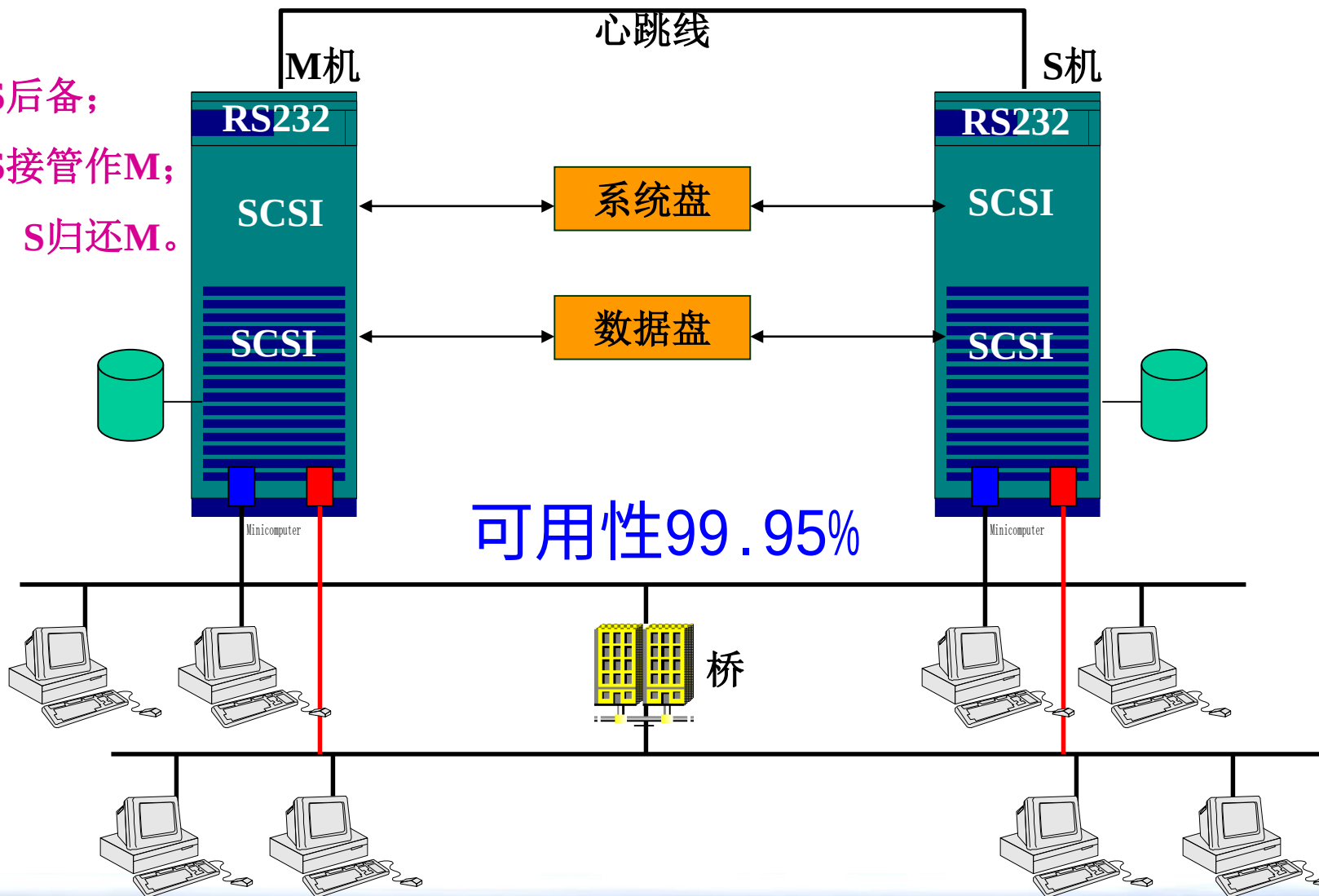


# 典型的容错应用

热备份，RAID技术， .....

# 重要！ 热备份（Hot-Standby）

- M运行，S后备；
- M故障，S接管作M；
- 原M修复，S归还M。



# RAID

廉价磁盘冗余阵列（Redundant Arrays of Inexpensive Disks），以多个低成本磁盘构成磁盘子系统，提供比单一硬盘更完备的可靠性和高性能。

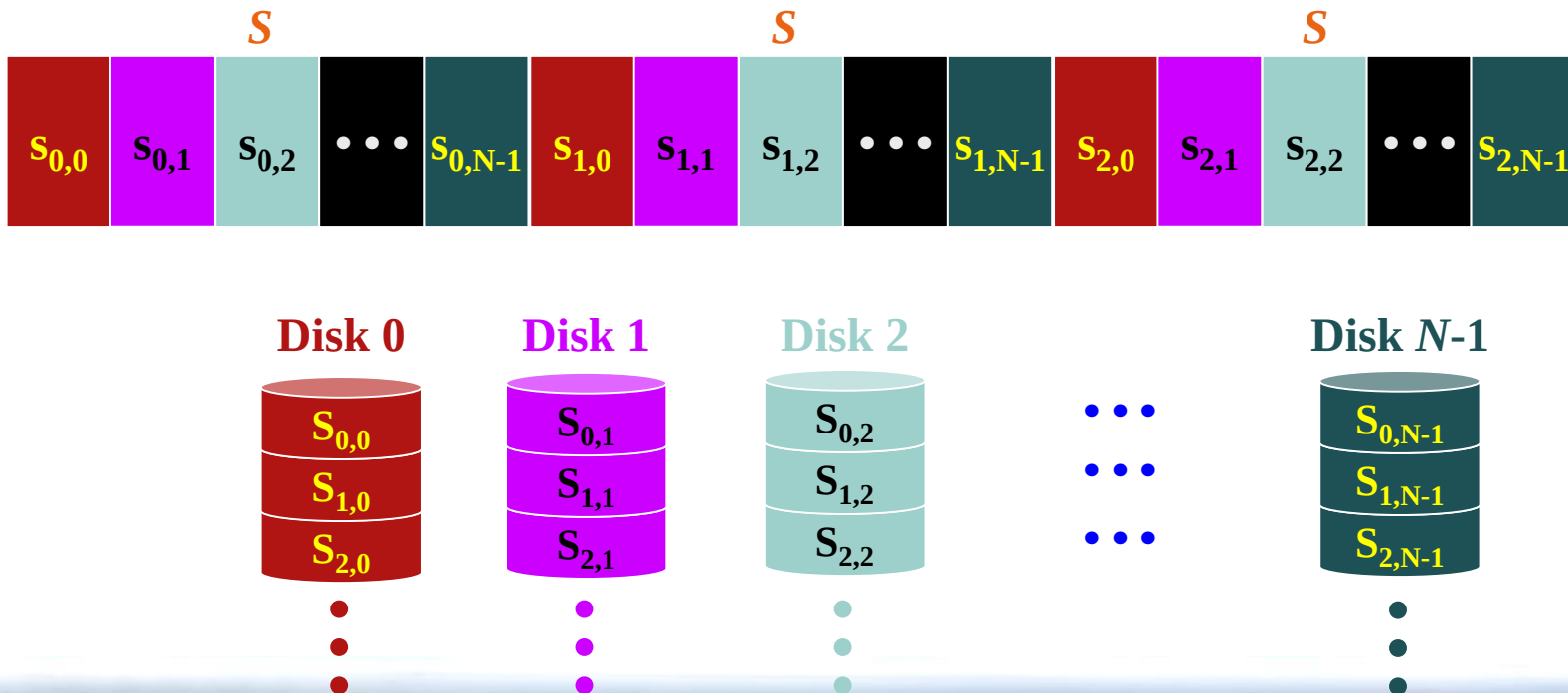
❖ **RAID**分级取决于三个因素：

- 分条**Striping**
- 数据镜像**Mirroring**
- 奇偶校验 **Parity**（ **Error Correction** ）



# RAID的分条——Exploit Parallelism

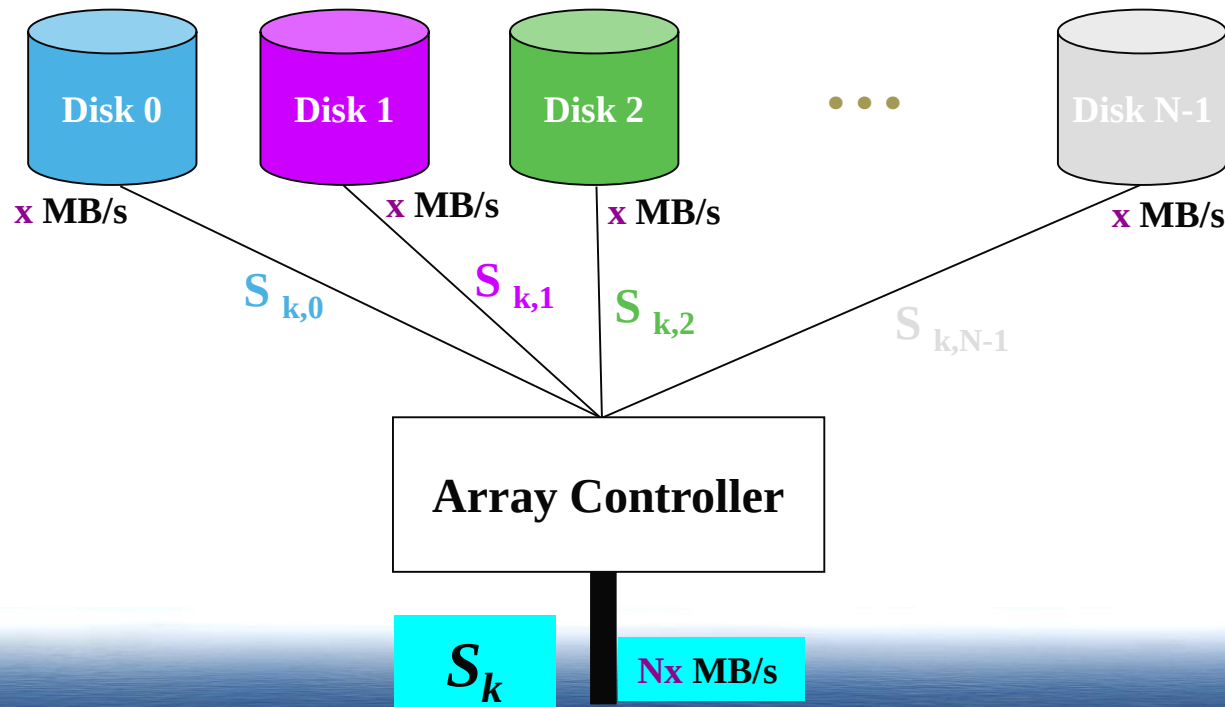
- ▶ *Stripe* the data across an array of disks
  - ▶ many alternative striping strategies possible
- ▶ Example: consider a big file striped across  $N$  disks
  - ▶ *stripe width* is  $S$  bytes
  - ▶ hence each *stripe unit* is  $S/N$  bytes
  - ▶ sequential read of  $S$  bytes at a time



# Performance Benefit

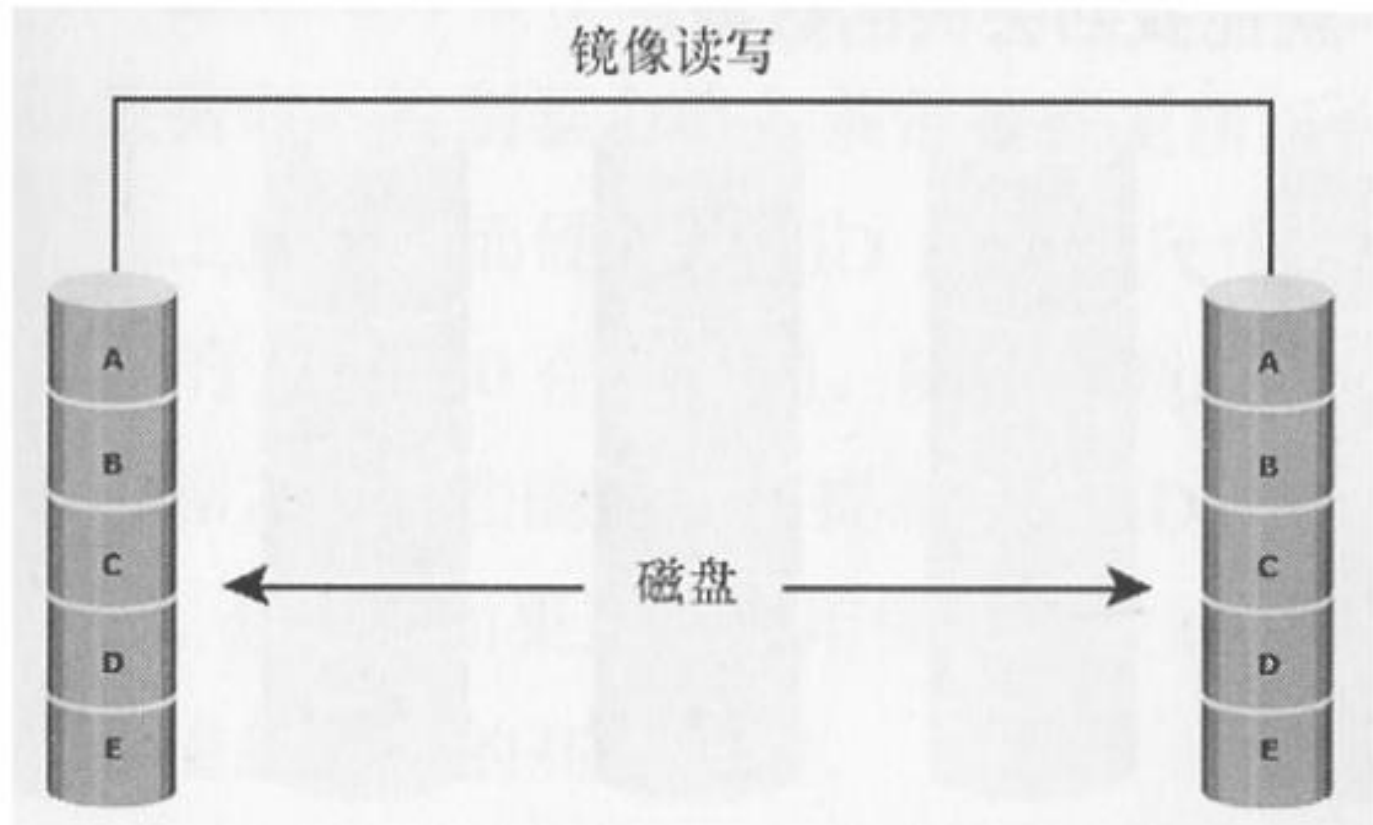
## *Sequential read or write of large file*

- application (or I/O buffer cache) reads in multiples of  $S$  bytes
- controller performs parallel access of  $N$  disks
- aggregate bandwidth is  $N$  times individual disk bandwidth
  - (assumes that disk is the bottleneck)



# RAID的分级——数据镜像

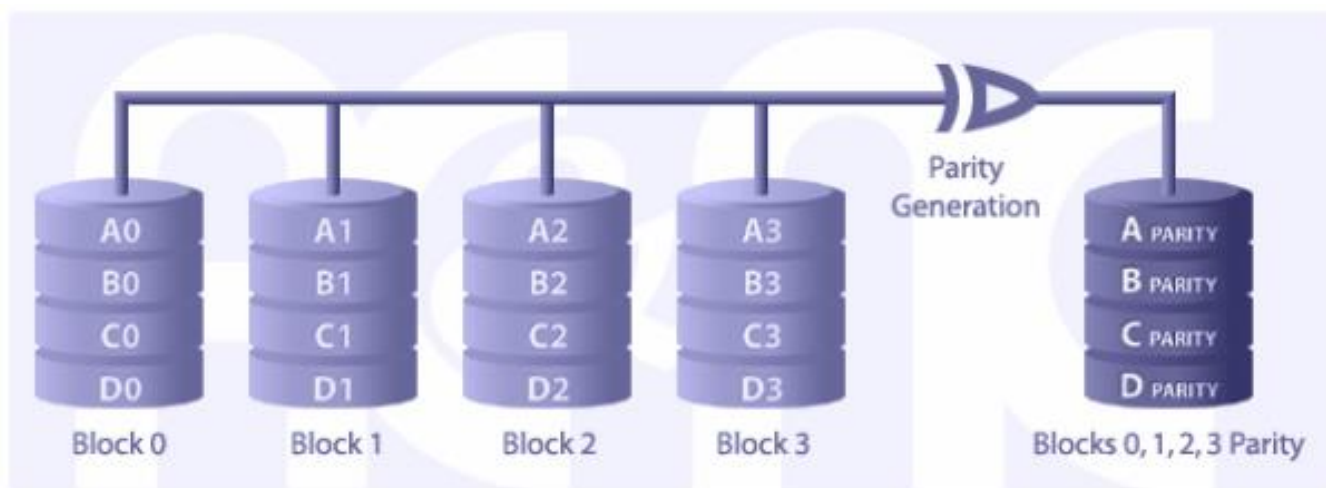
数据镜像（Mirroring）：将同一数据写在两块不同的硬盘上，从而产生该数据的两个副本。



# RAID的分级——ECC

ECC (Error Checking and Correcting) 的主要功能是“发现并纠正错误”。ECC纠错技术需要额外的空间来存储校正码，但其占用的位数跟数据的长度并非成线性关系。

通过数学方法而不是单纯重复写同样数据来实现数据保护。



e.g. 独立磁盘奇偶校验:校验信息单独存在磁盘上，一旦出现磁盘损坏，用校验值减去其它磁盘上对应位置的值，就能找回数据。

# 数据基带条阵列（RAID0）

**分块无校验型，无冗余存储。简单将数据分配到各个磁盘上，不提供真正容错性。带区化至少需要2个硬盘，可支持8/16/32个磁盘**

## □ 优点

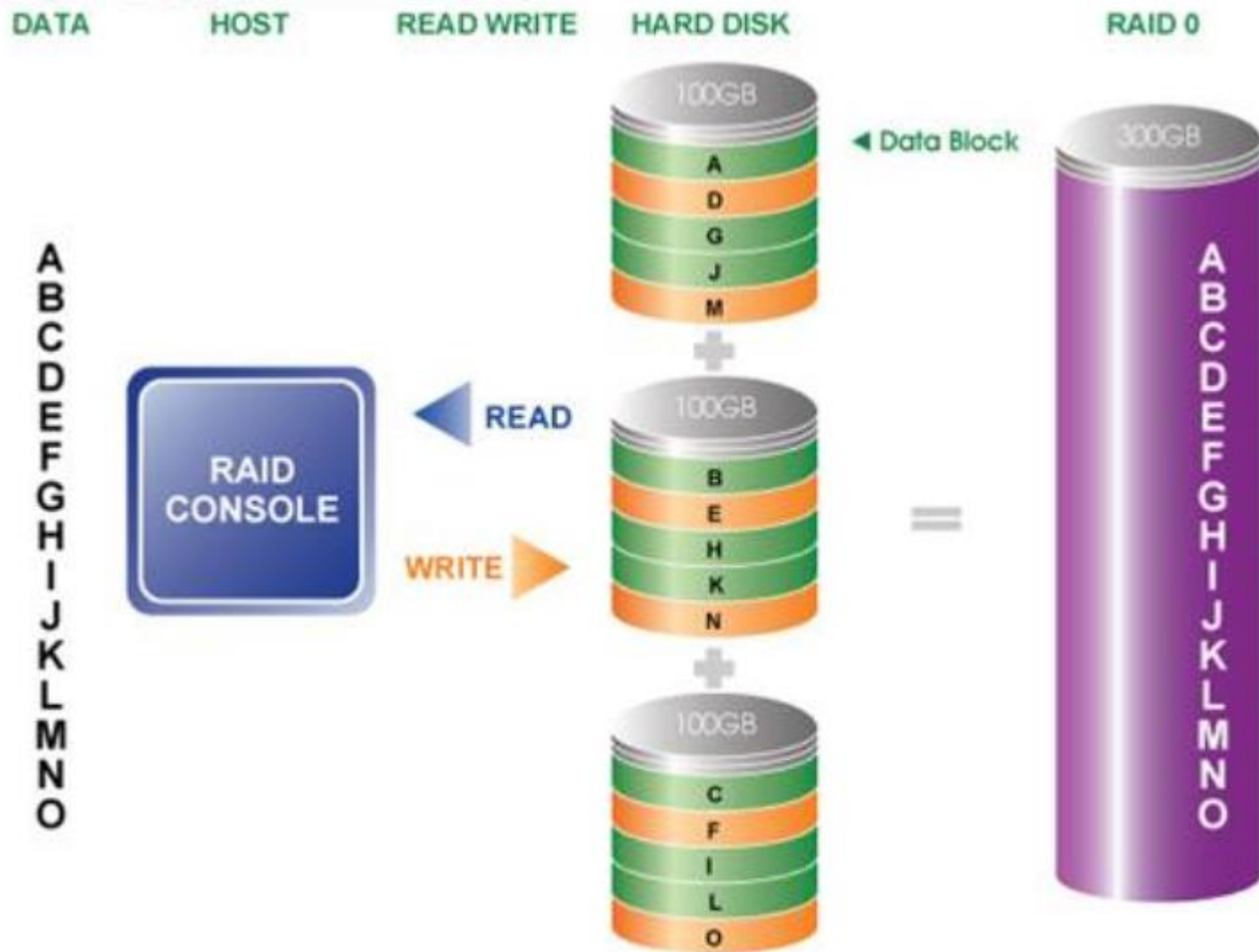
- 允许多个小区组合成一个大分
- 更好地利用磁盘空间，延长磁盘寿命
- 多个硬盘并行工作，提高了读写性能

## □ 缺点

- 不提供数据保护，任一磁盘失效，数据可能丢失，且不能自动恢复。

# RAID0示意图

输入数据流





# 磁盘镜像（RAID1）

每一组盘至少两台，数据同时以同样的方式写到两个盘上，两个盘互为镜像。磁盘镜像可以是分区镜像、全盘镜像。容错方式以空间换取，实施可以采用镜像或者双工技术。

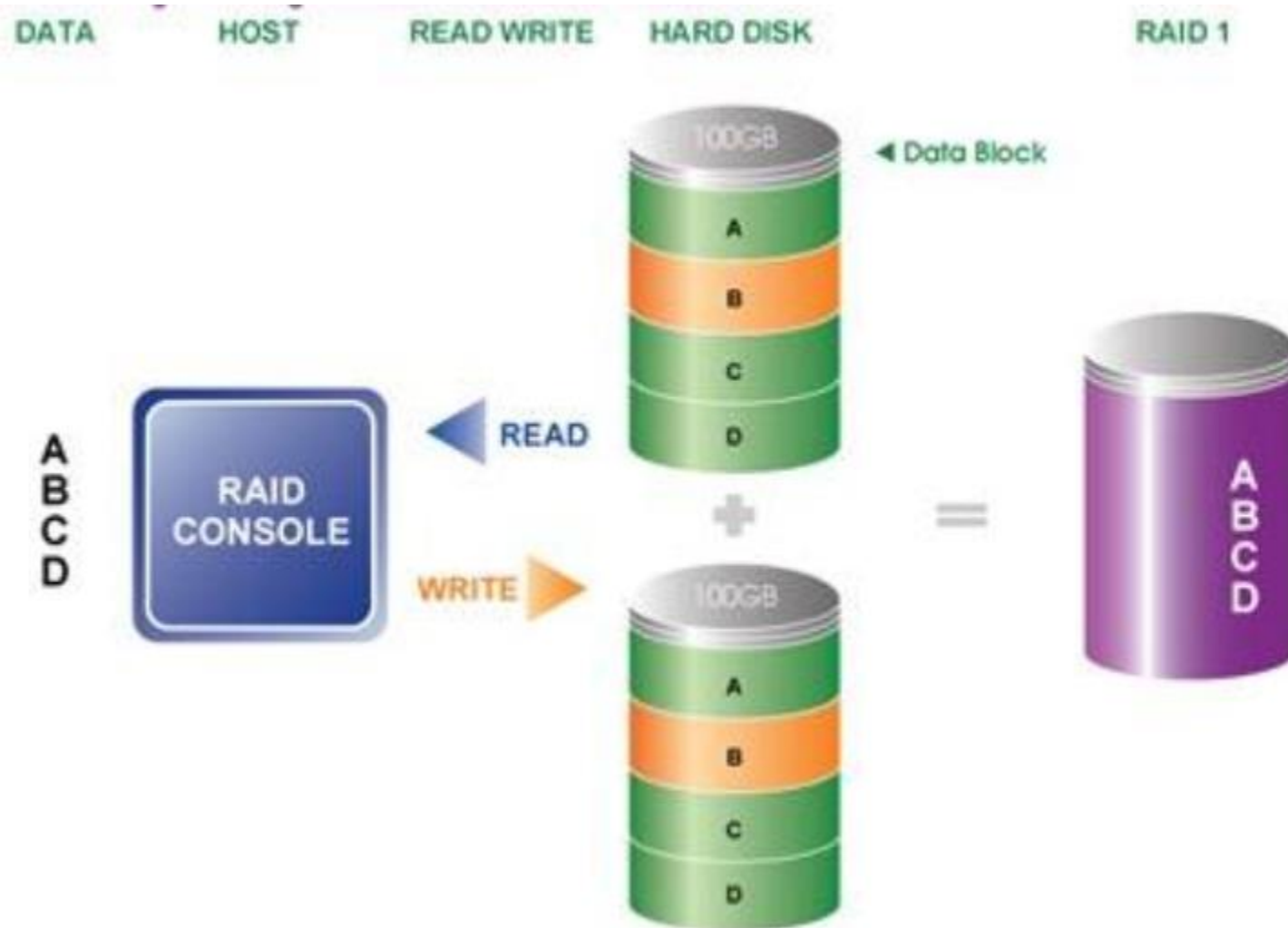
## □ 优点

- 可靠性高，策略简单，恢复数据时不必停机。

## □ 缺点

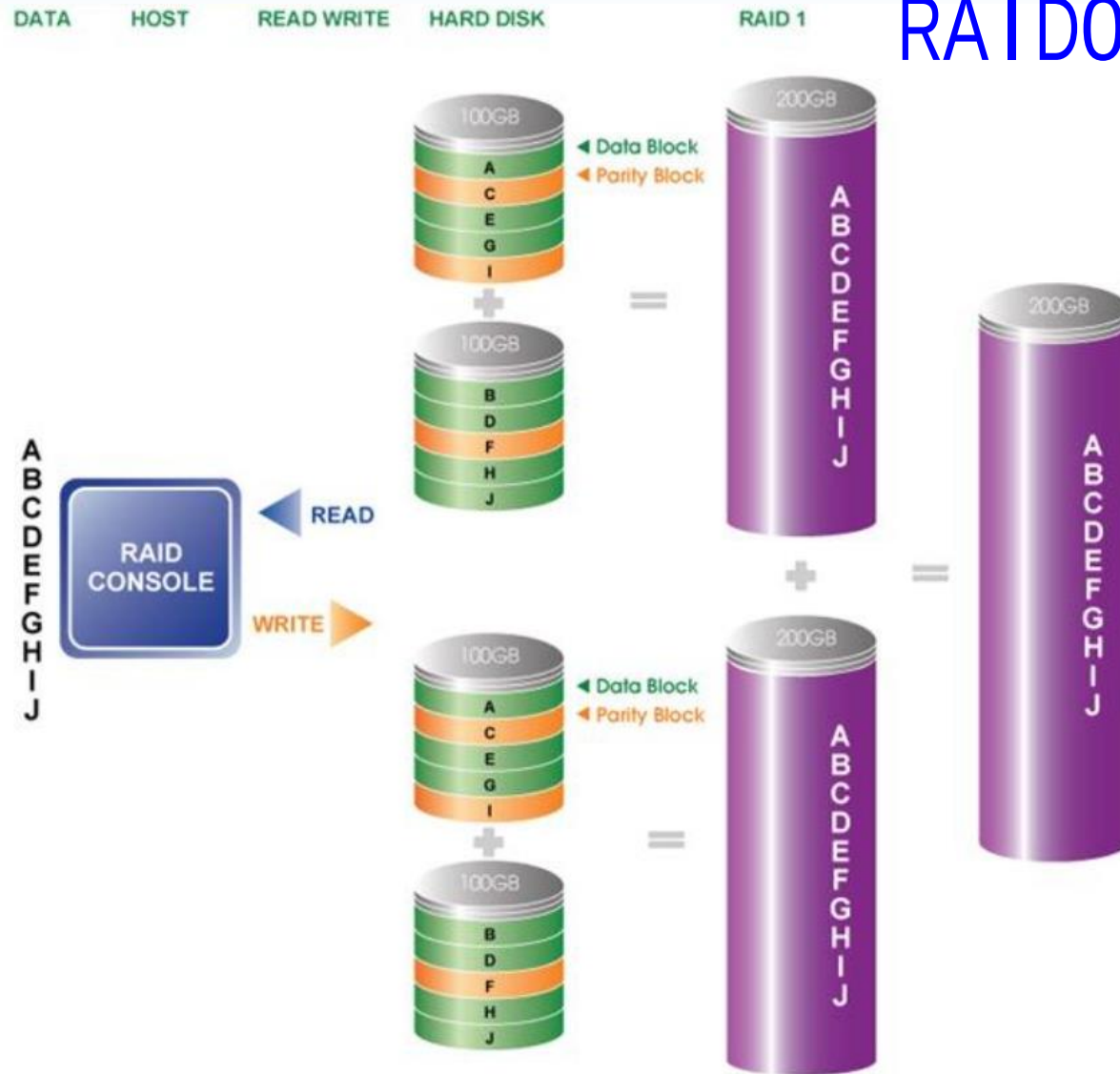
- 有效容量只有总容量的1/2，利用率50%。由于磁盘冗余，硬件开销较大，成本较高。

# RAID1



# RAID10

考试考：  
RAID01



# 循环奇偶校验阵列（RAID5）

与RAID4类似，但校验数据不固定在一个磁盘上，而是循环地依次分布在不同的磁盘上，也称块间插入分布校验。它是目前采用最多、最流行的方式，至少需要3个硬盘。

## □ 优点

- 校验分布在多个磁盘中，写操作可以同时处理；
- 为读操作提供了最优的性能；
- 一个磁盘失效，分布在其他盘上的信息足够完成数据重建。

## □ 缺点

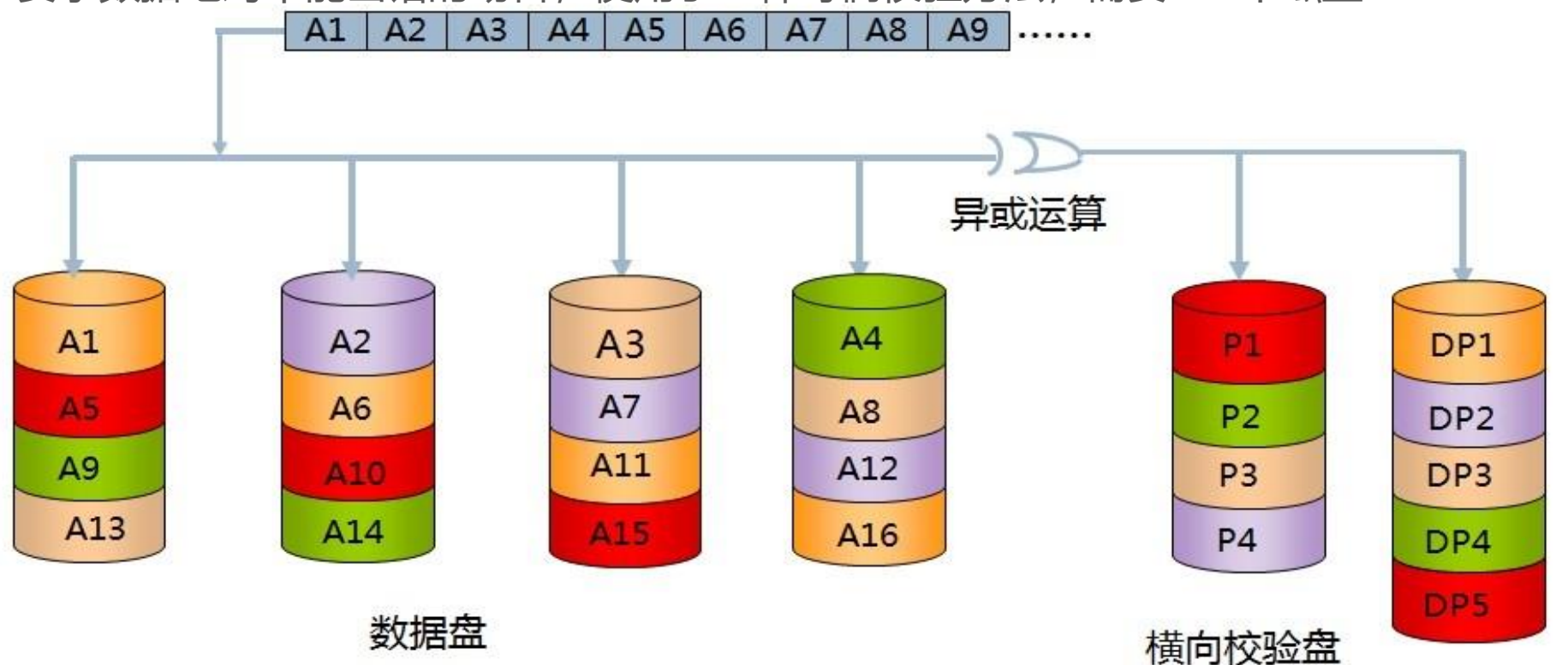
- 数据重建会降低读性能；
- 每次计算校验信息，写操作开销会增大，是一般存储操作时间的3倍。

# RAID5示意图



# 二维奇偶校验阵列（RAID6）

RAID6是指带有两种分布存储的检验信息的磁盘阵列，它是对RAID5的扩展，主要是用于要求数据绝对不能出错的场合，使用了二种奇偶校验方法，需要 $N+2$ 个磁盘



横向校验盘中P1—P4为各个数据盘中横向数据的校验信息

例： $P1 = A1 \text{ XOR } A2 \text{ XOR } A3 \text{ XOR } A4$

斜向校验盘中DP1—DP4为各个数据盘及横向校验盘的斜向数据的校验信息

例： $DP1 = A1 \text{ XOR } A6 \text{ XOR } A11 \text{ XOR } A16$



# RAID6 P+Q

- ❖ RAID6 P+Q会根据公式计算出P和Q的值，当有两个数据同时丢失时，仍可以计算出原数据

|     | 磁盘1  | 磁盘2  | 磁盘3  | 磁盘4  | 磁盘5  |
|-----|------|------|------|------|------|
| 条带1 | 数据1a | 数据1b | 数据1c | P1   | Q1   |
| 条带2 | 数据2d | 数据2e | P2   | Q2   | 数据2f |
| 条带3 | 数据3g | P3   | Q3   | 数据3h | 数据3i |
| 条带4 | P4   | Q4   | 数据4j | 数据4k | 数据4l |
| 条带5 | Q5   | 数据5m | 数据5n | 数据5o | P5   |

# 目前的RAID卡的厂商

- AMI



- Mylex



- Adaptec



- Intel



# Intel SCR31

- ❖ 64位 33MHz PCI2.2接口规范
- ❖ 基于64位的Intel i960 RN智能I/O处理器
- ❖ 32M Cache （可扩充到128M）
- ❖ 单个Ultra160 SCSI接口通道
- ❖ Intel® Integrated RAID管理软件
- ❖ 硬件XOR运算
- ❖ 支持RAID0、RAID1、RAID5、RAID0+1
- ❖ 高性能、高可靠性



# RAID的比较



(a) RAID 0: 非冗余条带



(b) RAID 1: 镜像磁盘



(c) RAID 2: 内存式错误纠正码



(d) RAID 3: 位交错奇偶校验



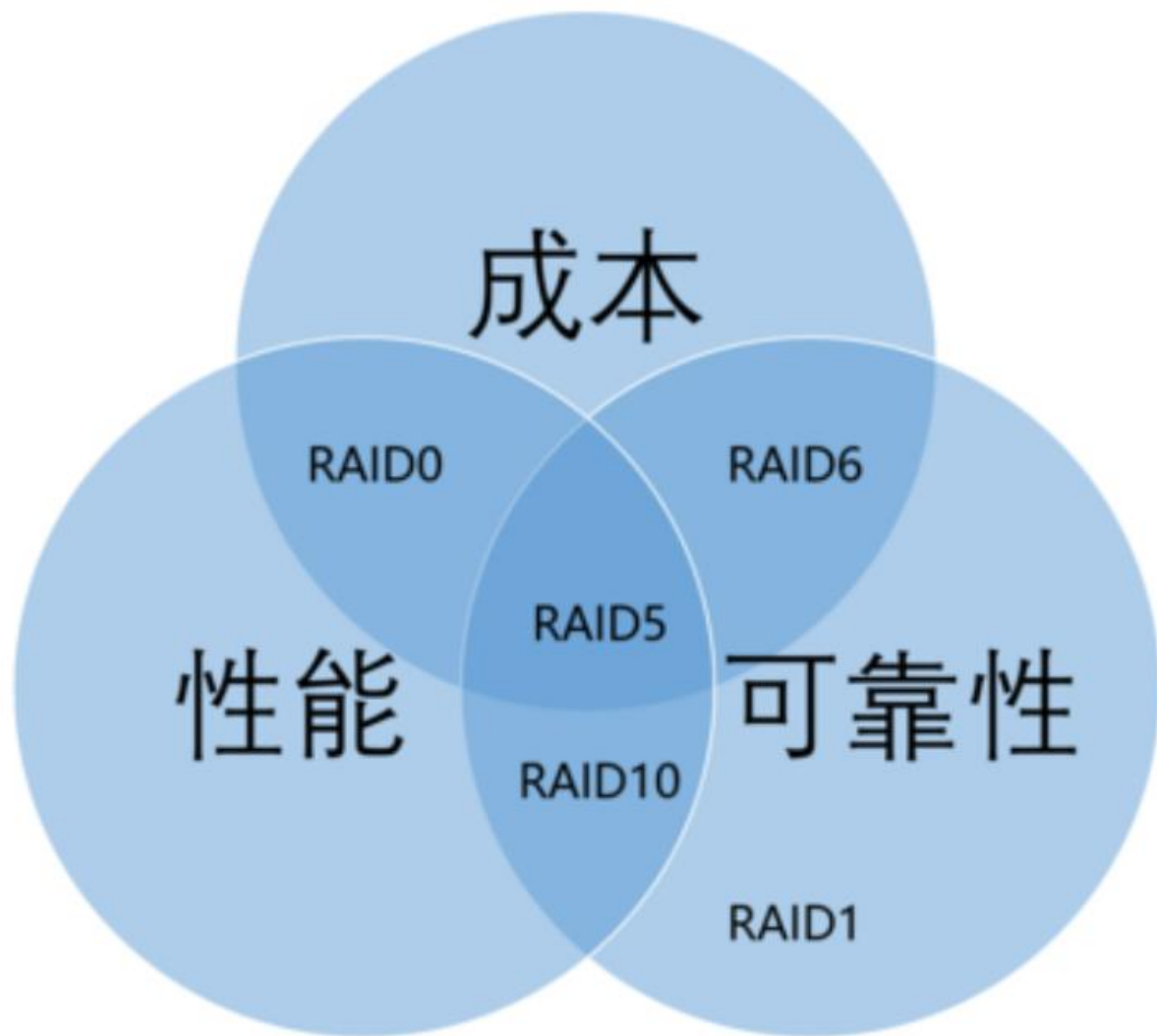
(e) RAID 4: 块交错奇偶校验



(f) RAID 5: 块交错分布式奇偶校验



(g) RAID 6: P + Q 冗余



# 如果损坏的硬盘超过2块怎么办？

## 删除码（Erasure Coding）

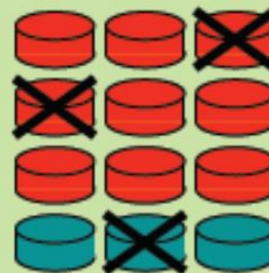
A technique that lets you take  $n$  storage devices:



Encode them onto  $m$  additional storage devices:



And have the entire system be resilient to up to  $m$  device failures:



Erasure code是云存储的核心技术，最初诸如Hadoop、GFS、CEPH等都采用的是n-way replication来做冗余，但是这样会带来极大的成本开销，因此几乎各大公司都在用erasure code替代n-way replication。



# Erasure Codes

以更小的数据冗余度获得更高数据可靠性



# Eurasure Coding

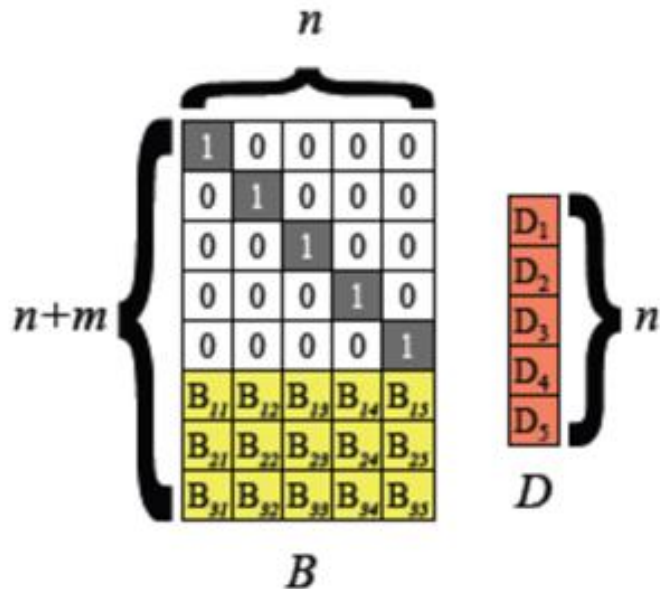
Erasure Code (EC)，即纠删码，是一种前向错误纠正技术（Forward Error Correction, FEC），主要应用在网络传输中避免包的丢失，存储系统利用它来提高存储可靠性。相比多副本复制而言，纠删码能够以更小的数据冗余度获得更高数据可靠性，但编码方式较复杂，需要大量计算。

EC的定义：Erasure Code是一种编码技术，它可以将 $n$ 份原始数据，增加 $m$ 份数据，并能通过 $n+m$ 份中的任意 $n$ 份数据，还原为原始数据。即如果有任意小于等于 $m$ 份的数据失效，仍然能通过剩下的数据还原出来。

# Reed-Solomon Codes

erasure code有很多种，其中RS code是最基本的一种。RS codes是基于 Galois Field 的一种编码算法，在RS codes中使用 $GF(2^w)$ ，其中 $2^w \geq n+m$ 。

- Codes are based on linear algebra.
  - Next, define an  $(n+m) \times n$  “Distribution Matrix”  $B$ , whose first  $n$  rows are the identity matrix:



RS codes定义了一个 $(n+m) \times n$ 的分发矩阵 (Distribution Matrix)

Plank J S. Erasure Codes for Storage Systems: A Brief Primer[J]. login: the Usenix Magazine, 2013, 38(6): 44-50.

# Reed-Solomon Codes

因此，对每一段的 $n$ 份数据，我们都可以通过 $B * D$  得到：

- Codes are based on linear algebra.
  - $B * D$  equals an  $(n+m) * 1$  column vector composed of  $D$  and  $C$  (the coding words):

$$\begin{array}{c}
 \overbrace{\begin{array}{|c|c|c|c|c|} \hline 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 1 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 \\ \hline 0 & 0 & 0 & 0 & 1 \\ \hline B_{11} & B_{12} & B_{13} & B_{14} & B_{15} \\ \hline B_{21} & B_{22} & B_{23} & B_{24} & B_{25} \\ \hline B_{31} & B_{32} & B_{33} & B_{34} & B_{35} \\ \hline \end{array}}^n \\
 \left. \begin{array}{c} \\ \\ \\ \\ \\ \\ \\ \end{array} \right\} n+m \\
 \underbrace{\hspace{10em}}_B
 \end{array}
 *
 \begin{array}{|c|} \hline D_1 \\ \hline D_2 \\ \hline D_3 \\ \hline D_4 \\ \hline D_5 \\ \hline \end{array}
 =
 \begin{array}{|c|} \hline D_1 \\ \hline D_2 \\ \hline D_3 \\ \hline D_4 \\ \hline D_5 \\ \hline C_1 \\ \hline C_2 \\ \hline C_3 \\ \hline \end{array}
 \begin{array}{l} \left. \begin{array}{c} \\ \\ \\ \\ \\ \end{array} \right\} D \\ \left. \begin{array}{c} \\ \\ \end{array} \right\} C \end{array}$$

# Reed-Solomon Codes

假如D1、D4、C2 失效，那么我们可以同时从矩阵B和B\*D中，去掉响应的行，得到下面的等式：

- Suppose  $m$  nodes fail.
- To decode, we create  $B'$  by deleting the rows of  $B$  that correspond to the failed nodes.
- You'll note that  $B' * D$  equals the survivors.

$$\begin{array}{ccccc}
 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 \\
 B_{11} & B_{12} & B_{13} & B_{14} & B_{15} \\
 B_{31} & B_{32} & B_{33} & B_{34} & B_{35}
 \end{array}
 \begin{array}{c}
 * \\
 \\
 \\
 \\
 \\
 \end{array}
 \begin{array}{c}
 D_1 \\
 D_2 \\
 D_3 \\
 D_4 \\
 D_5
 \end{array}
 =
 \begin{array}{c}
 D_2 \\
 D_3 \\
 D_5 \\
 C_1 \\
 C_3
 \end{array}$$

$B'$ 
 $D$ 
Survivors

# Reed-Solomon Codes

- Now, invert  $B'$ :
- And multiply both sides of the equation by  $B'^{-1}$

$$\begin{array}{c}
 \begin{array}{|c|c|c|c|c|}
 \hline
 \text{Yellow} & \text{Red} & \text{Blue} & \text{Purple} & \text{Green} \\
 \hline
 1 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 0 & 0 \\
 \text{Pink} & \text{Green} & \text{Pink} & \text{Orange} & \text{Brown} \\
 0 & 0 & 1 & 0 & 0 \\
 \hline
 \end{array} \\
 B'^{-1}
 \end{array}
 *
 \begin{array}{c}
 \begin{array}{|c|c|c|c|c|}
 \hline
 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 \\
 B_{11} & B_{12} & B_{13} & B_{14} & B_{15} \\
 B_{31} & B_{32} & B_{33} & B_{34} & B_{35} \\
 \hline
 \end{array} \\
 B'
 \end{array}
 *
 \begin{array}{c}
 D_1 \\
 D_2 \\
 D_3 \\
 D_4 \\
 D_5
 \end{array}
 =
 \begin{array}{c}
 \begin{array}{|c|c|c|c|c|}
 \hline
 \text{Yellow} & \text{Red} & \text{Blue} & \text{Purple} & \text{Green} \\
 \hline
 1 & 0 & 0 & 0 & 0 \\
 0 & 1 & 0 & 0 & 0 \\
 \text{Pink} & \text{Green} & \text{Pink} & \text{Orange} & \text{Brown} \\
 0 & 0 & 1 & 0 & 0 \\
 \hline
 \end{array} \\
 B'^{-1}
 \end{array}
 *
 \begin{array}{c}
 D_2 \\
 D_3 \\
 D_5 \\
 C_1 \\
 C_3
 \end{array}
 \text{Survivors}$$

# Reed-Solomon Codes

- Now, invert  $B'$ :
- And multiply both sides of the equation by  $B'^{-1}$
- Since  $B'^{-1} * B' = I$ , You have just decoded  $D$ !

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 1 |

$I$

$$\begin{array}{c} D_1 \\ D_2 \\ D_3 \\ D_4 \\ D_5 \end{array} * \begin{array}{ccccc} \text{Yellow} & \text{Orange} & \text{Blue} & \text{Purple} & \text{Green} \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ \text{Pink} & \text{Green} & \text{Pink} & \text{Orange} & \text{Brown} \\ 0 & 0 & 1 & 0 & 0 \end{array} = \begin{array}{c} D_2 \\ D_3 \\ D_5 \\ C_1 \\ C_3 \end{array}$$

$B'^{-1}$       Survivors



# A summary of Reed-Solomon Codes

上面的推导并不严谨，存在一些问题：

- (1)  $B'$ 是否一定存在可逆矩阵？
- (2) Distribution Matrix -  $B$ 如何求得？
- (3) 每次都用乘法，CPU的开销会很大。

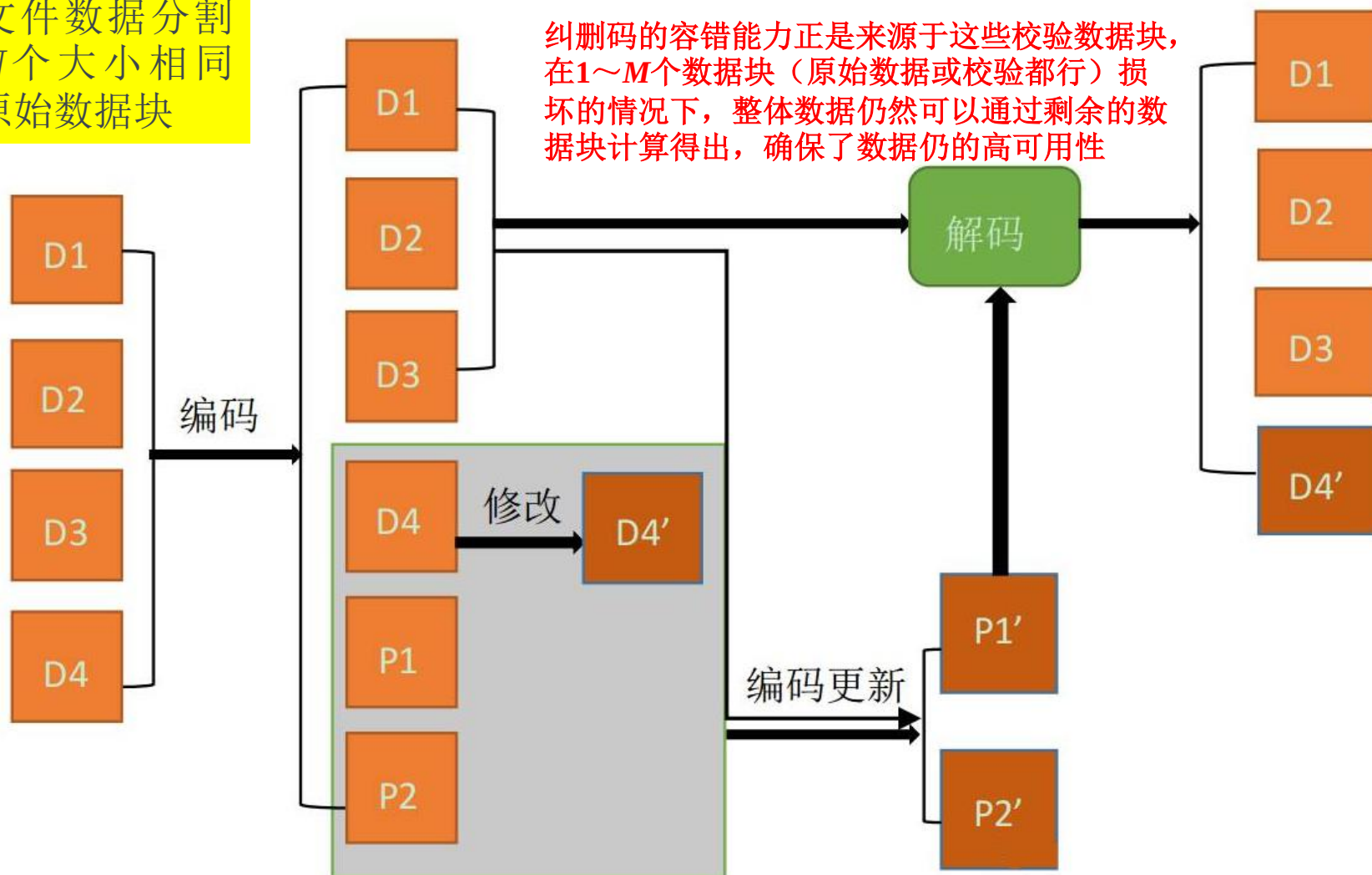
An  $(n + m) \times n$  Vandermonde matrix is defined as follows:

针对上面问题，有如下解决方案：利用 Vandermonde 矩阵的特性，即其任意的子方阵均为可逆方阵。

$$\begin{bmatrix} 1 & a_1^1 & a_1^2 & \dots & a_1^{n-1} \\ 1 & a_2^1 & a_2^2 & \dots & a_2^{n-1} \\ 1 & a_3^1 & a_3^2 & \dots & a_3^{n-1} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & a_{n+m}^1 & a_{n+m}^2 & \dots & a_{n+m}^{n-1} \end{bmatrix}$$

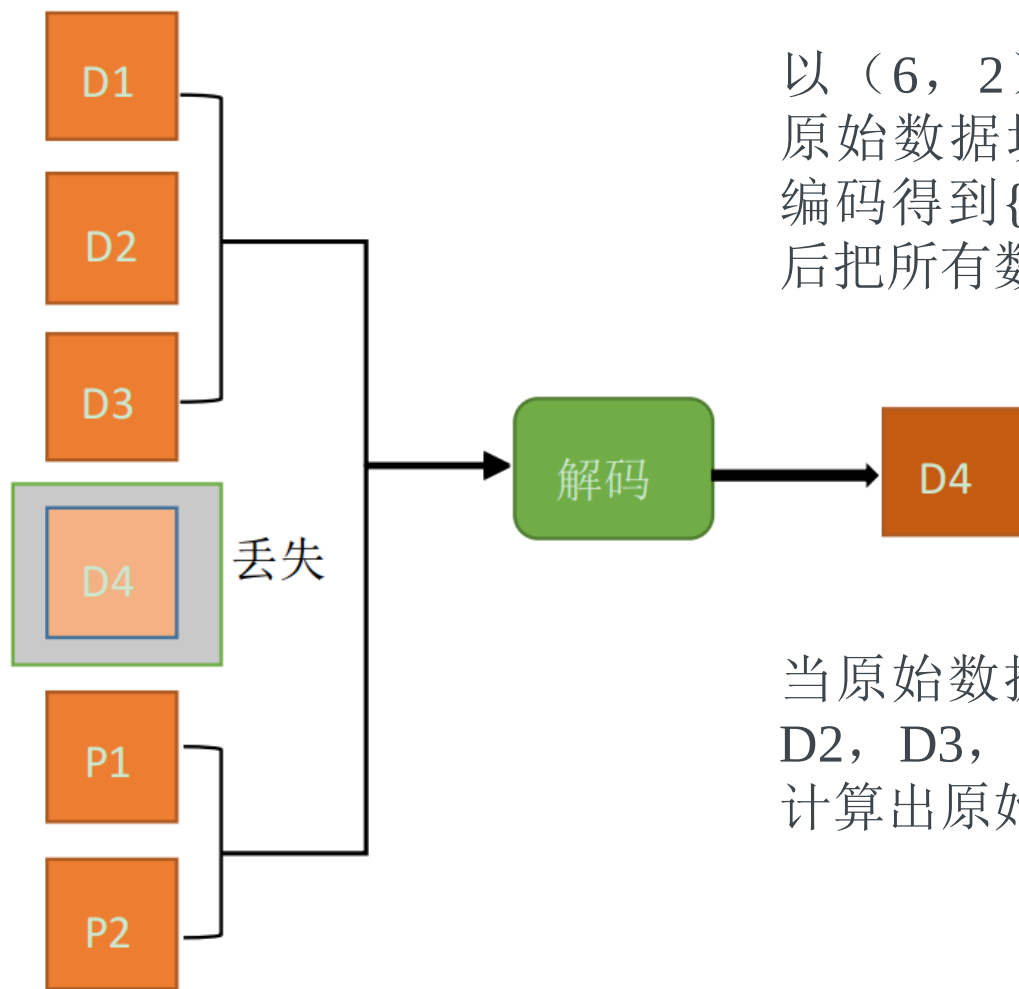
# 纠删码用于存储

将文件数据分割成 $N$ 个大小相同的原始数据块



编码生成 $M$ 个大小相同的校验数据块（原始数据块和校验数据块大小相同），最后将原始数据块和校验数据块分别存储在不同的位置，如不同的磁盘、不同的存储节点等。

# 纠删码存储的数据分块恢复

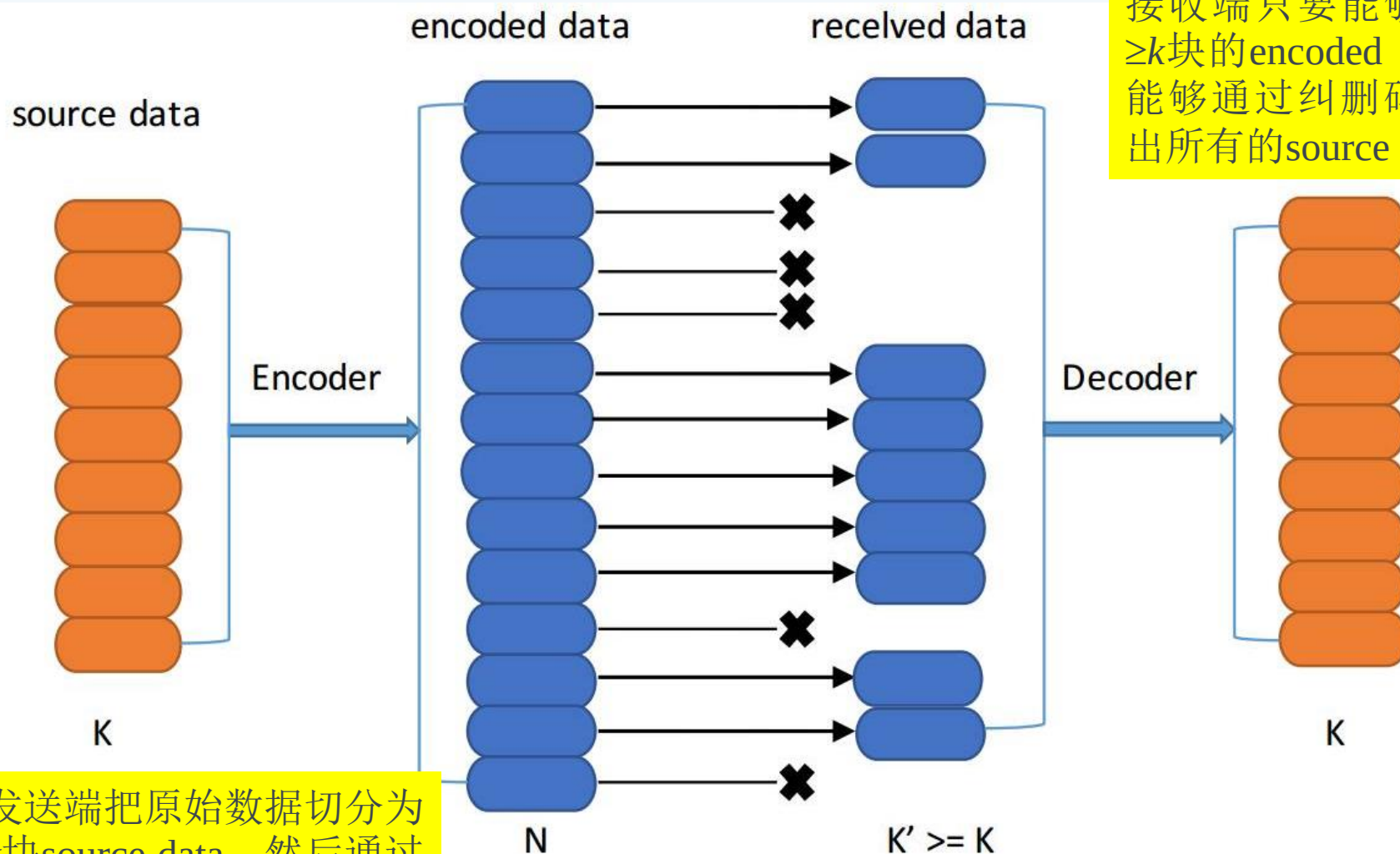


以  $(6, 2)$  RS 码为例，文件划分为4个原始数据块{D1, D2, D3, D4}，经过编码得到{P1, P2}两个校验数据块，然后把所有数据块统一存储。

当原始数据分块D4丢失时，可根据{D1, D2, D3, P1, P2}中任意4个数据块解码计算出原始数据块D4。

在  $(6, 2)$  RS 码为例的纠删码存储中，其冗余度为2，即最多可以同时丢失2块数据块，当丢失的数据块个数大于2时，丢失数据块就不可恢复了。

# 纠删码用于通信传输



接收端只要能够接收到  $\geq k$  块的 encoded data，就能够通过纠删码 decoder 出所有的 source data。

发送端把原始数据切分为  $k$  块 source data，然后通过纠删码编码生成  $n$  块 encoded data，最后统一向服务端传输。

让各数据段之间产生一定的联系，然后统一把所有信号段向接收端发送；在传输过程中可能有部分信号丢失

# 基于EC的开源实现技术



# Intel ISA-L库

Intel ISA-L (Intelligent Storage Acceleration Library)，即英特尔智能存储加速库，是英特尔公司开发的一套专门用来加速和优化基于英特尔架构 (Intel Architecture, IA) 存储的lib库，可在多种英特尔处理器上运行，能够最大程度提高存储吞吐量、安全性和灵活性，并减少空间使用量。

- 该库可加速RS码的计算速度，通过使用AES-NI、SSE、AVX、AVX2等指令集来提高计算速度，从而提高编解码性能。
- 还可在存储数据可恢复性、数据完整性、数据安全性以及加速数据压缩等方面提供帮助，并提供了一组高度优化的功能函数，包括RAID函数、纠删码函数、CRC（循环冗余检查）函数、缓冲散列（MbH）函数、加密函数、压缩函数等。



# Jerasure库

Jerasure是美国田纳西大学Plank教授开发的C/C++纠删码函数库，提供Reed-Solomon和Cauchy Reed-Solomon两种编码算法的实现。Jerasure有1.2和2.0两个常用版本，Jerasure 2.0为目前的最新版本，可借助Intel SSE指令集来加速编解码，相比1.2版本有较大的提升。

Jerasure库分为5个模块，每个模块包括一个头文件和实现文件。

| 头文件/实现文件                  | 功能                                                                                                                                  |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| galois.h/galois.c         | 提供了伽罗华域算术运算                                                                                                                         |
| jerasure.j/jerasure.c     | 为绝大部分纠删码提供的核心函数，它只依赖galois模块。这些核心函数支持基于矩阵的编码与解码、基于位矩阵的编码与解码、位矩阵变换、矩阵转置和位矩阵转置                                                        |
| reedsol.h/reedsol.c       | 支持RS编/解码和优化后的RS编码                                                                                                                   |
| cauchy.h/Cauchy.c         | 支持柯西RS编解码和最优柯西RS编码                                                                                                                  |
| liberation.h/liberation.c | 支持Liberation RAID-6编码、Blaum-Roth编码和Liberation RAID-6编码。其中，Liberation是一种最低密度的MDS编码。这三种编码采用位矩阵来实现，其性能优于现在有RS码和EVENODD，在某种情况下也优于RDP编码。 |

# Intel ISA vs. Jerasure



**Intel ISA-L库**

二者均能加速RS码的计算速度。其中，ISA-L 库对于加速RS码的计算速度效果更好，是目前业界最佳。ISA-L 之所以速度快，主要有两点，一是由于Intel 谙熟汇编优化之道，ISA-L直接使用汇编代码；二是因为它将整体矩阵运算搬迁到汇编中进行。但这导致了汇编代码的急剧膨胀，令人望而生畏。另外，ISA-L未对Vandermonde 矩阵做特殊处理，而是直接拼接单位矩阵作为其编码矩阵，因此在某些参数下会出现编码矩阵线性相关的问题。



**Jerasure2.0库**

相较ISA-L 库对于加速 RS 码的计算速度效果略差，但是Jerasure2.0库在存储应用中仍具有一些ISA-L库所没有的优势，如Jerasure 2.0使用 C 语言封装后的指令，让代码更加的友好。另外Jerasure2.0 不仅仅支持  $GF(2^8)$  有限域的计算，其还可以进行  $GF(2^4) - GF(2^{128})$  之间的有限域。并且除了 RS 码，还提供了Cauchy Reed-Solomon code (CRS码) 等其他编码方法的支持。且在工业应用之外，其学术价值也非常高，是目前使用最为广泛的编码库之一，开源的Ceph分布式存储系统就是使用Jerasure库作为默认的纠删码库。

# Summary of The Class



- 容错的概念

即使存在故障也可保证可靠性

- 容错的研究内容

故障检测，故障屏蔽，冗余，软件容错，

- 容错的应用

RAID

- Erasure code